Лабораторна робота № 6. Збірка на Vite

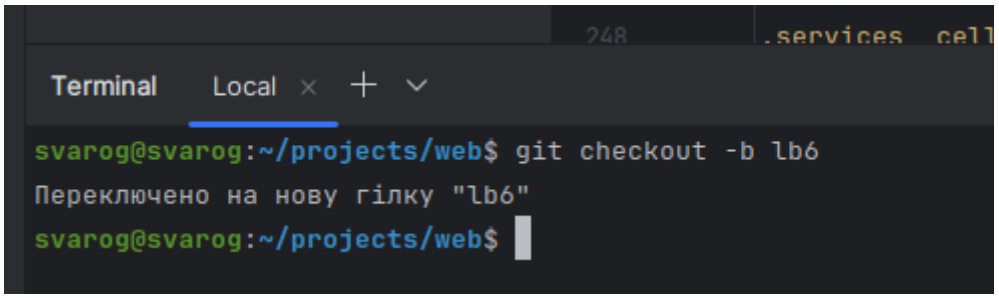
Мета: навчитися збирати проєкт на Vite, набути практичних навичок написання SCSS, навчитися розбивати HTML на окремі partials.

Завдання

1. За основу взяти попередню лабораторну роботу (виконати git pull, якщо не зробили це у попередній лабораторній);
2. Виконати пункти 1-3 ходу роботи;
3. У звіті відобразити:
    a. якщо робота захищається здобувачем особисто: посилання на git-репозиторій, посилання на задеплоєний сайт, скріншоти сайту (локальний та задеплоєний), скріншоти виконання команд dev/build/preview та теку dist;
    b. якщо робота не захищається: відобразити всі кроки ходу роботи з власними коментарями, посилання на git-репозиторій, посилання на задеплоєний сайт, скріншоти виконання команд dev/build/preview та теку dist, відповіді на КП.

Хід роботи

1. Створюємо нову гілку на основі існуючої (main) за допомогою команди **git checkout -b lb6**.



2. У цій лабораторній перед нами стоять наступні задачі: перевести проєкт на Vite, конвертувати стилі в SCSS, розбити повторюванні html-блоки на окремі partials.

Почнемо із реструктуризації проєкту, для початку створимо package.json та встановимо Vite.

Спочатку перевіряємо версію Node (якщо не встановлено, то інструкція у “Додаткові матеріали до лабораторної роботи №6”)

```
svarog@svarog:~/projects/web$ node -v
v22.21.0
svarog@svarog:~/projects/web$
```

package.json створюємо за допомогою команди **npm init -y** (-y дозволяє пропустити діалоговий режим та створити файл з параметрами за замовчуванням).

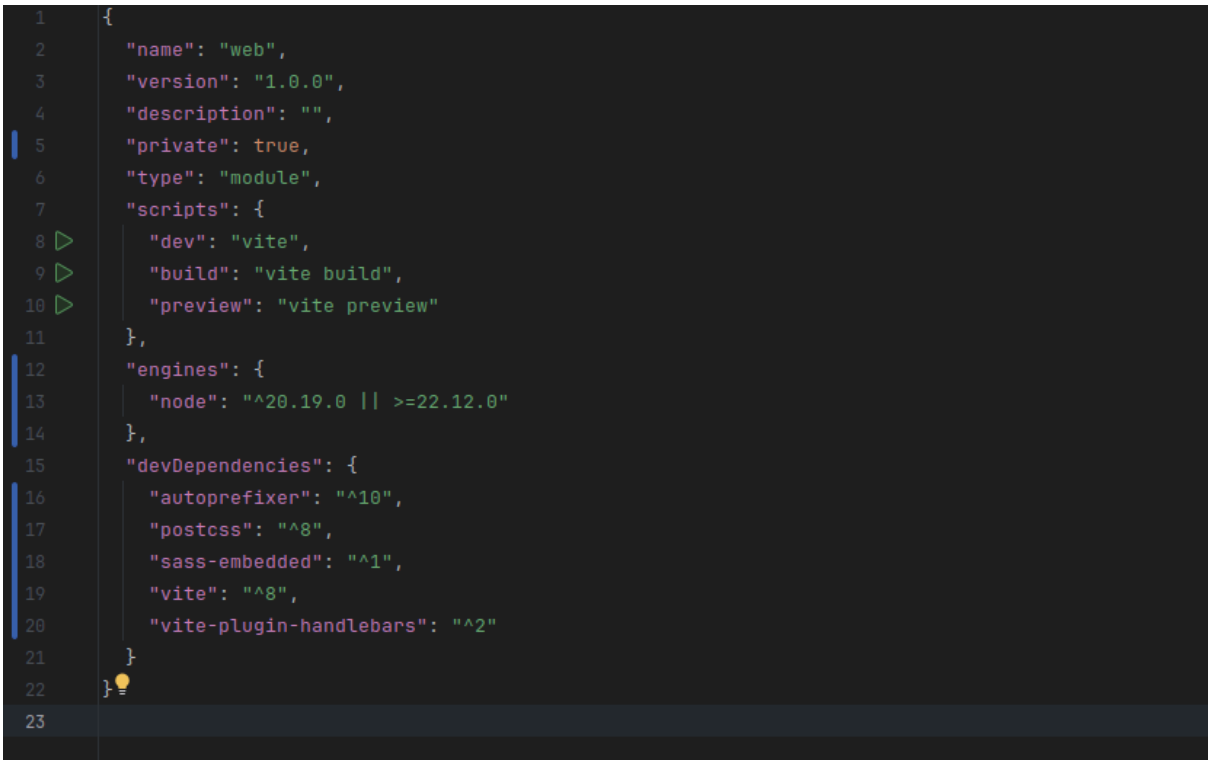
```
svarog@svarog:~/projects/web$ npm init -y
Wrote to /home/svarog/projects/web/package.json:

{
  "name": "web",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Далі встановлюємо devDependencies (тобто ті, які використовуються тільки при розробці) сам Vite, а також відразу Saas, PostCSS та Autoprefixer.

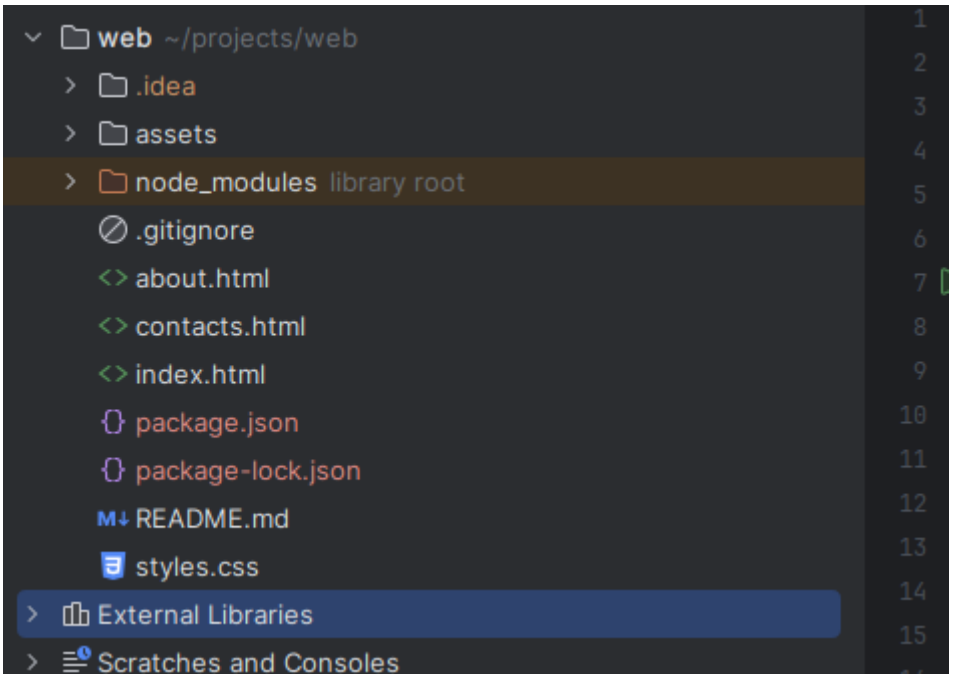
```
svarog@svarog:~/projects/web$ npm i -D vite@^8 sass-embedded postcss autoprefixer
```

Перевіряємо, що наші залежності підтягнулися в devDependencies package.json.

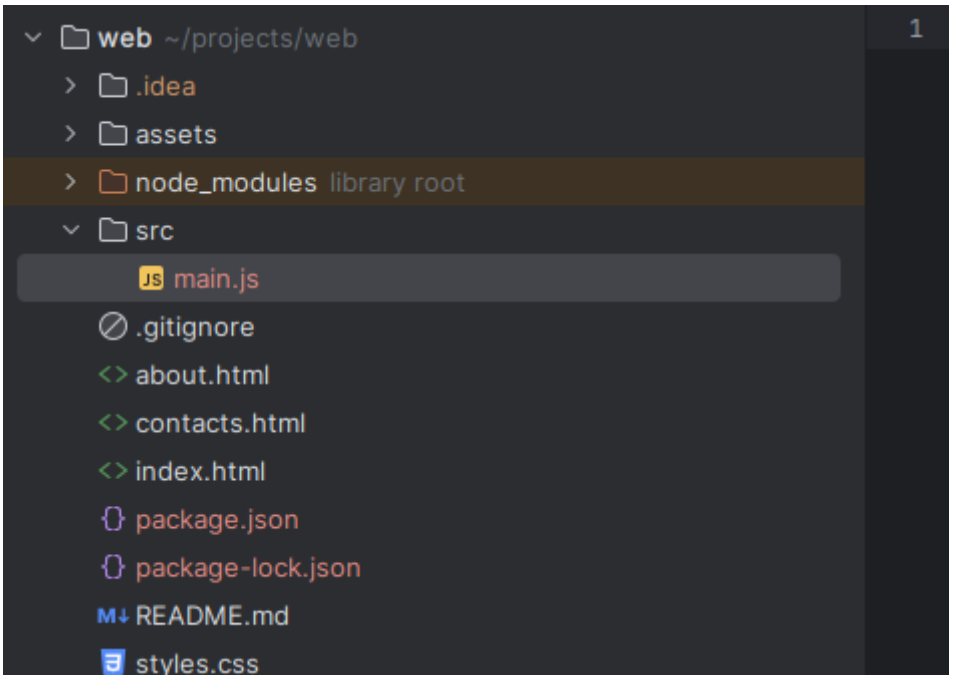


```
{
  "name": "web",
  "version": "1.0.0",
  "description": "",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "engines": {
    "node": "^20.19.0 || >=22.12.0"
  },
  "devDependencies": {
    "autoprefixer": "^10",
    "postcss": "^8",
    "sass-embedded": "^1",
    "vite": "^8",
    "vite-plugin-handlebars": "^2"
  }
}
```

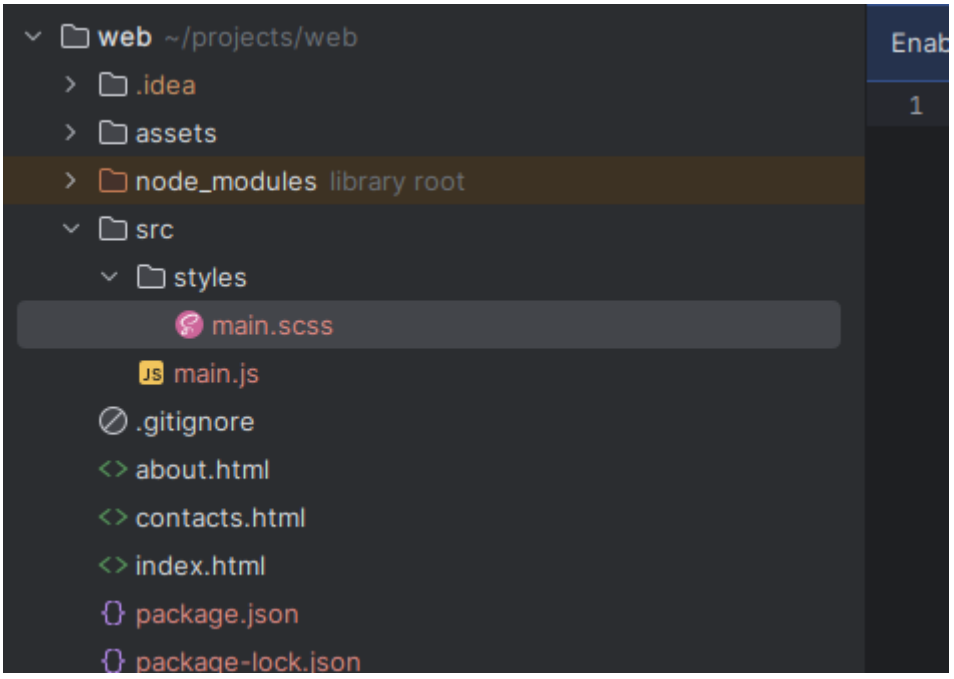
Також у нашому проєкті згенерувався файл **package-lock.json** та тека **node_modules/**.



Далі створюємо теку src і у ній файл main.js. Це наша точка входу.



У теці src створюємо теку styles із файлом main.scss.



Імпортуємо файл стилів у main.js

main.js

```
1  import './styles/main.scss';
```

Підключаємо main.js в усі сторінки (<link href="styles.css"> прибираємо, так як ми його вже і не будемо використовувати, і стилі тепер підключаються через main.js).

Демонструю на прикладі index.html, у інших робимо за аналогією (головне не приберіть нічого зайвого і пам’ятаємо, що підключаємо перед </body>).

Для того щоб підключити використовуємо тег <script></script>.

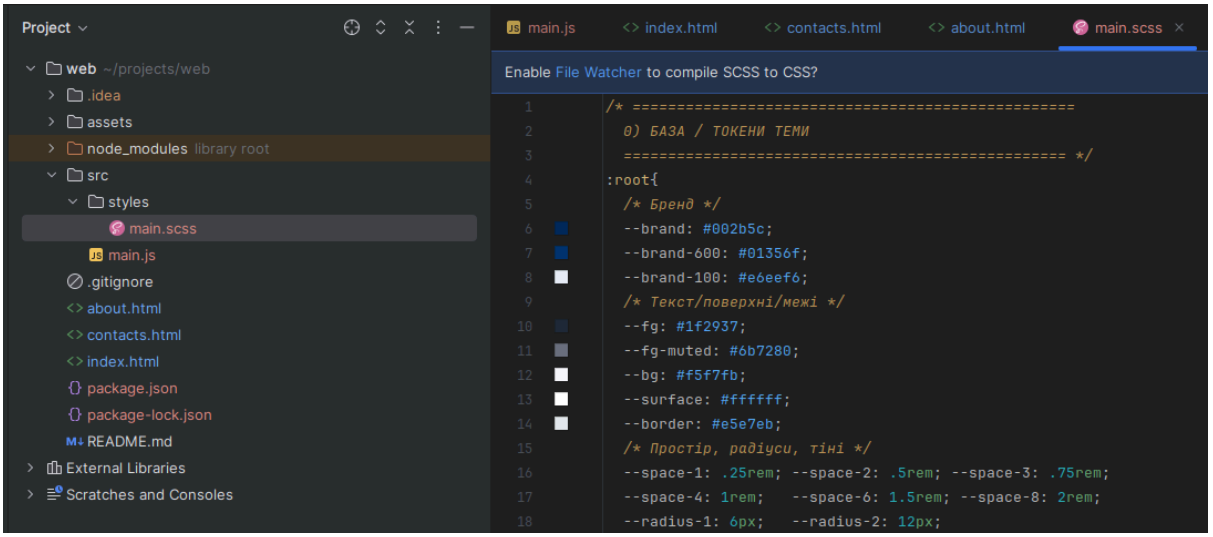
main.js <> index.html

```
2   <html lang="uk">
10  <body>
11  <div class="layout">
118     <footer class="footer">
119         <div class="footer__inner">
132                     <a class="footer__link" href="tel:+380501112233">+3
133                 </li>
134                 <li class="footer__contacts-item">
135                     <span class="footer__label">Адреса:</span>
136                     <span>Запоріжжя, вул. Університетська, 66Б</span>
137                 </li>
138             </ul>
139         </address>
140     </div>
141 </footer>
142 </div>
143 <script type="module" src="/src/main.js"></script>
144 </body>
145 </html>
```

Для зручності у файл package.json властивість scripts прописуємо скрипти для запуску Vite.

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
},
```

Так як ми вже не будемо використовувати styles.css (можна видалити), то стилі з нього тимчасово перенесемо до main.scss (так-так, ми це можемо зробити, бо пам’ятаємо, що scss конвертується в css, тому і жодних проблем у використані нативного css немає).

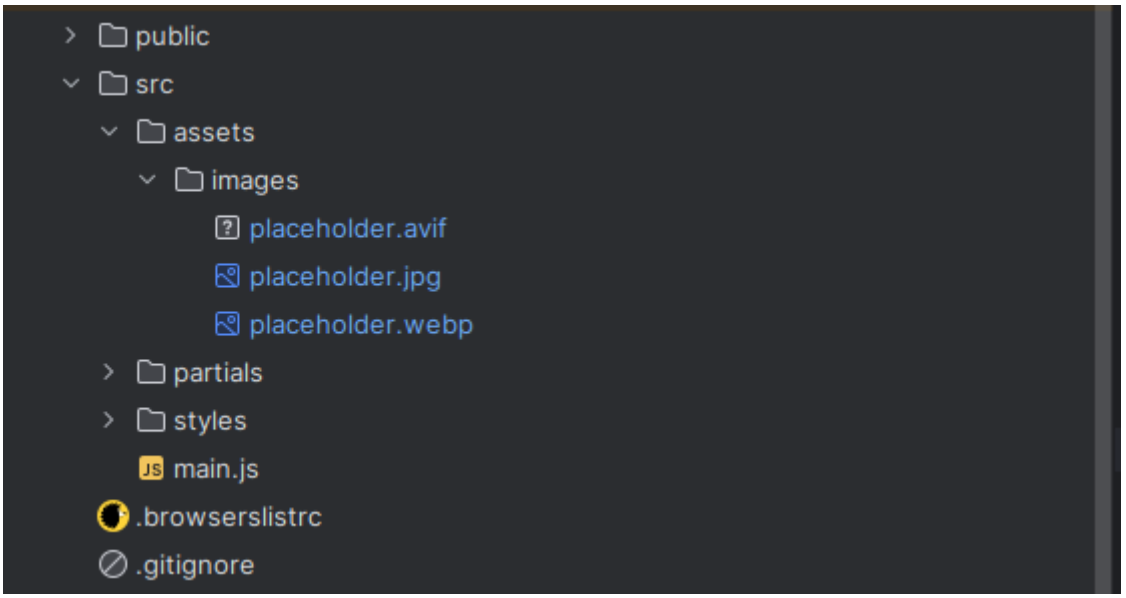


Наступним кроком створюємо структуру для зберігання різного роду статики. Ось тут цікавий момент, що змішує задати питання: “А куди переносити зображення нашого проєкту?”. Давай з цим розберемося, це нескладно, якщо пам’ятати декілька основних правил:

1) у public кладемо ту статику, яка повинна потрапити до збірки без обробки з боку Vite. Це, наприклад, favicon.ico, robots.txt або інші файли, для яких потрібно зберегти незмінне ім’я та шлях. Файли з public під час збірки просто копіюються до кореня dist - Vite не додає до їх назв хеш і не включає їх до свого asset pipeline;
2) у src кладемо ресурси, які є частиною нашого проєкту та повинні оброблятися Vite. Це стосується не лише файлів, які імпортуються через JavaScript або CSS. Vite також обробляє посилання на ресурси безпосередньо в HTML, зокрема в <img>, <picture> та <source>. Під час production-збірки такі файли потрапляють до dist/assets, а Vite може змінити їх назви, додавши хеш для правильного кешування.

Тому в нашому випадку зображення сайту переносимо до src/assets/images, оскільки вони є частиною інтерфейсу та повинні проходити через Vite під час збірки. У public залишаємо тільки ті ресурси, які дійсно потрібно копіювати без обробки.

Увага!!! Ахтунг!! При перенесенні файлів або тек IDE може автоматично змінити шляхи у коді. Це зручно, але все одно перевіряємо результат. Також пам’ятаємо: якщо файл знаходиться у public, слово public у URL не пишемо.



Ось так повинен виглядати шлях

```
<article class="news-card">
    <figure class="news-card__media media">
        <picture>
            <source type="image/avif" srcset="/src/assets/images/placeholder.avif">
            <source type="image/webp" srcset="/src/assets/images/placeholder.webp">
            <img
                    src="/src/assets/images/placeholder.jpg"
                    alt="Фото зі студентського заходу"
                    class="media__img"
                    width="640" height="360"
                    loading="lazy" decoding="async" />
        </picture>
    </figure>
    <h3 class="news-card__title">Старт навчального веб-проєкту</h3>
```

Тепер потрібно перевірити, що ми все зробили правильно. Для цього запускаємо dev-сервер за допомогою скрипта **npm run dev**

```
svarog@svarog:~/projects/web$ npm run dev

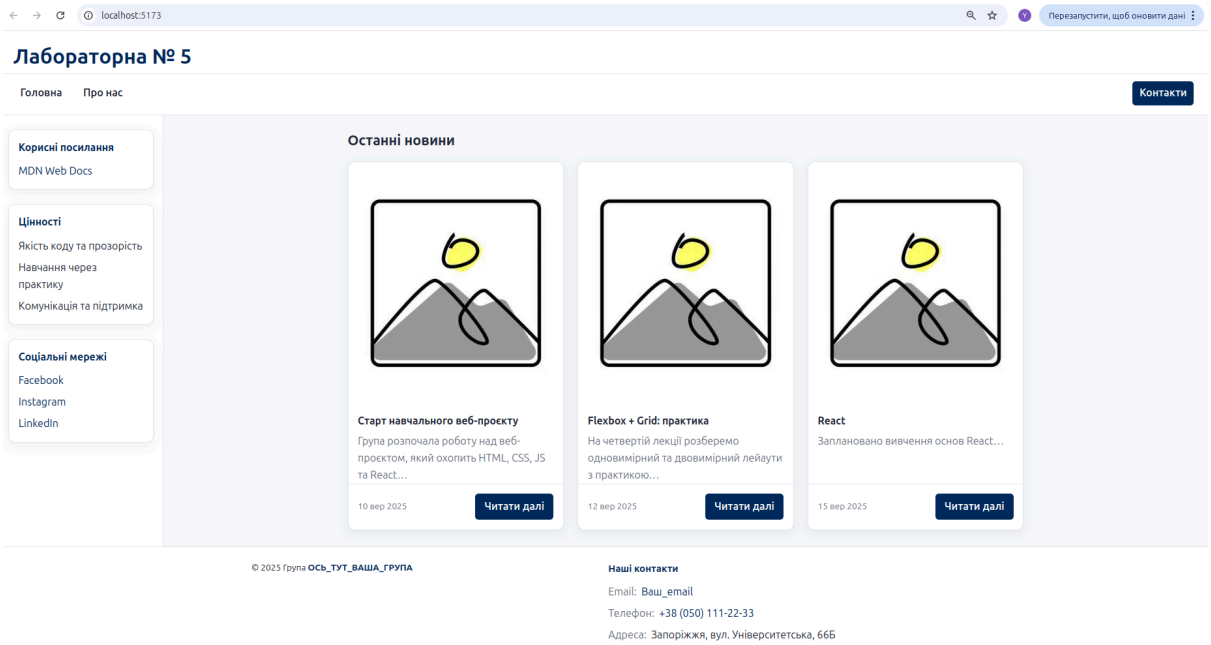
> web@1.0.0 dev
> vite


  VITE v8.3.4  ready in 169 ms

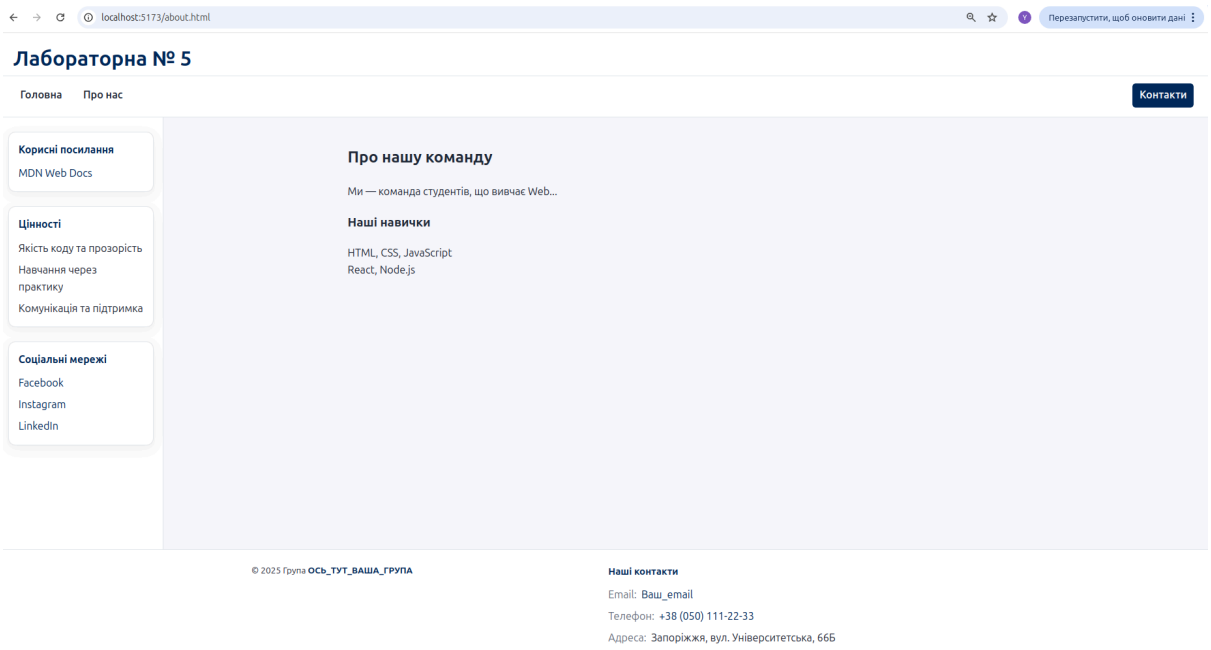
  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Переходимо за адресою та перевіряємо функціонал та відображення

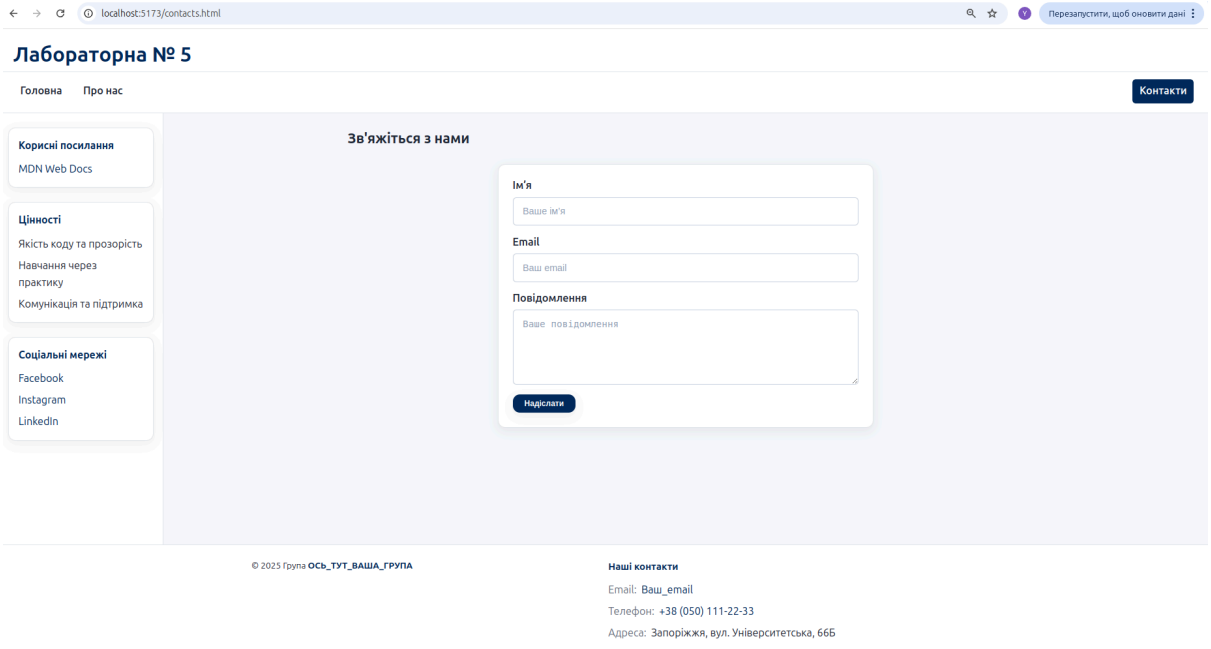
index.html



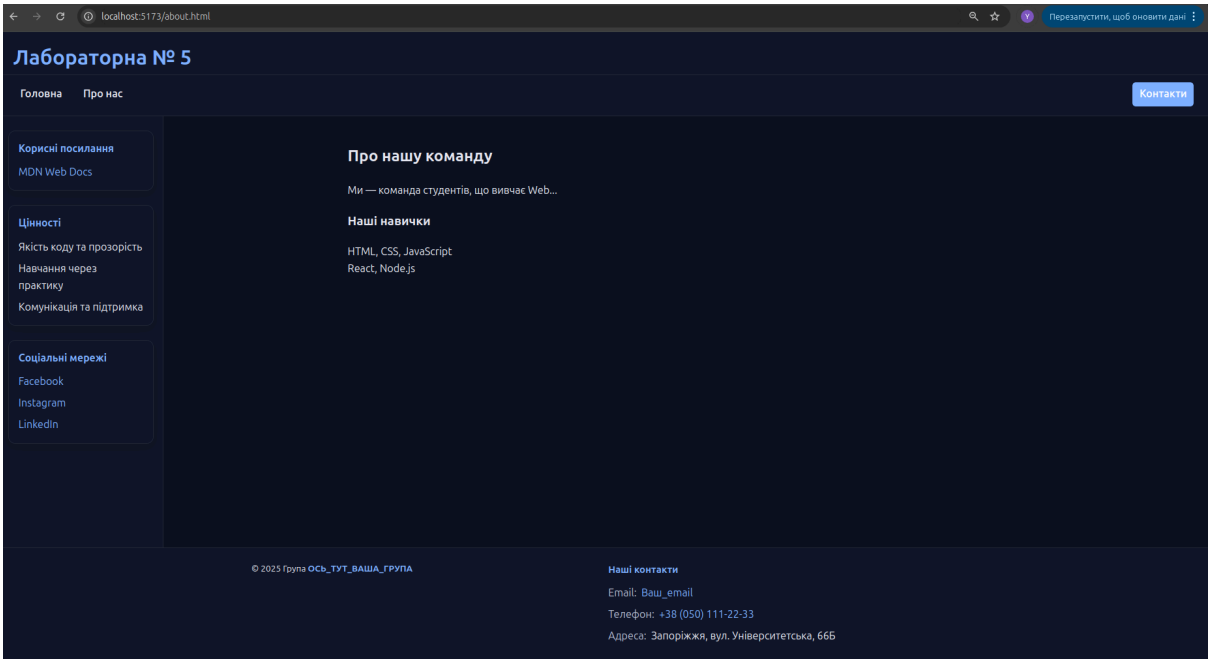
about.html



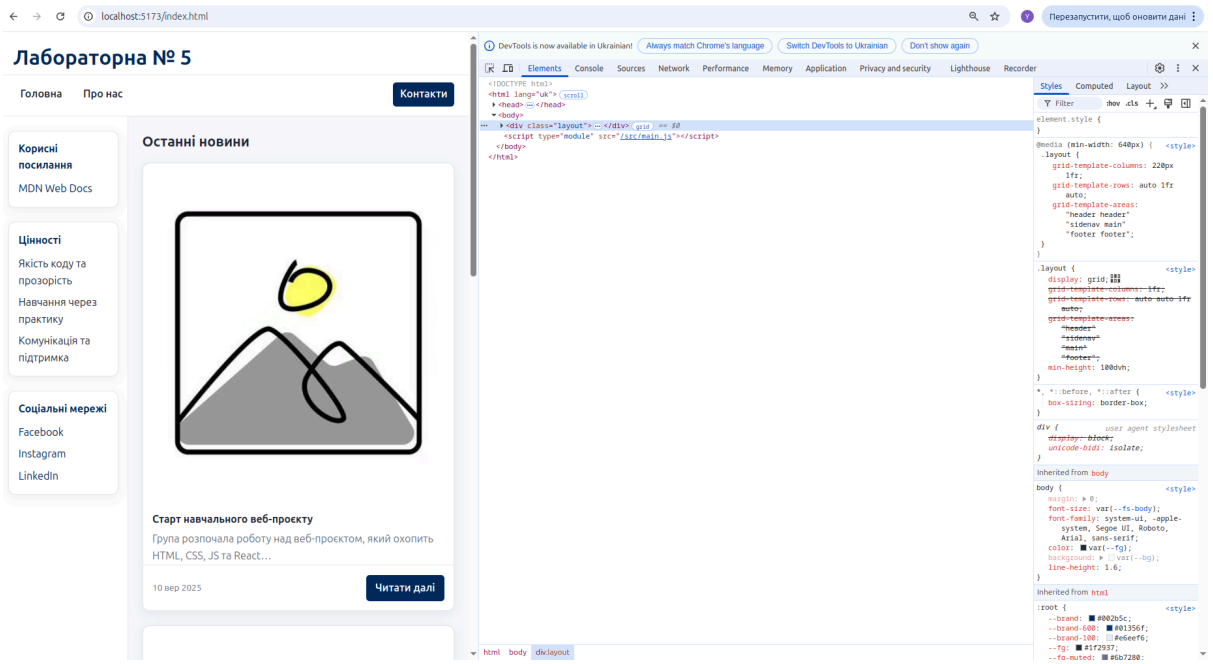
contacts.html



Перевіряємо відклик на системну тему



І адаптив



Також потрібно перевірити, що збірка проєкту проходить без помилок, для цього створимо файл vite.config.js з вказанням всіх entry.

```
export default defineConfig( config: {  no usages  Yaroslav Kryvyi *
    input: {
        main: resolve(import.meta.dirname, 'index.html'),
        about: resolve(import.meta.dirname, 'about.html'),
        contacts: resolve(import.meta.dirname, 'contacts.html'),
    },
    resolve: { alias: { '@': resolve(import.meta.dirname, 'src') } },
```

І запускаємо npm run build

```
svarog@svarog:~/projects/web$ npm run build

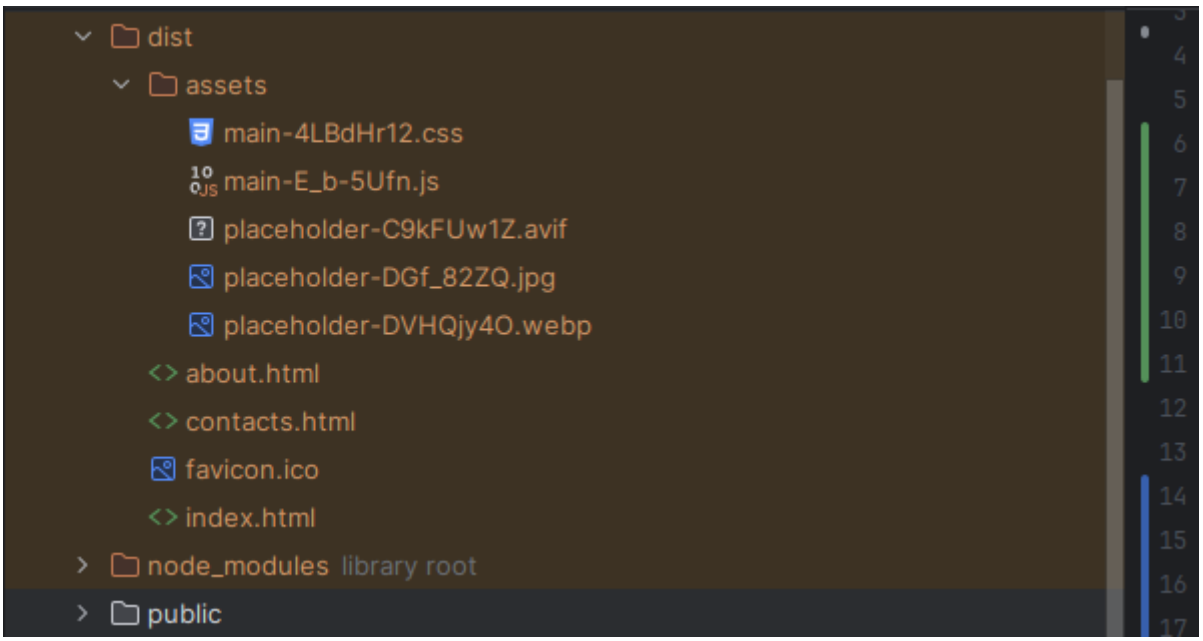
> web@1.0.0 build
> vite build

vite v8.3.4 building client environment for production...
✓ 8 modules transformed.
computing gzip size...
dist/assets/placeholder-DVHQjy4O.webp   4.44 kB
dist/about.html                         4.45 kB | gzip: 1.46 kB
dist/contacts.html                      4.98 kB | gzip: 1.55 kB
```

Після npm run build створюється тека dist. Vite генерує production-файли JavaScript/CSS та оброблені assets; їхні назви зазвичай містять content hash для ефективного кешування. Файли з public копіюються до dist без змін назв.

Не слід перевіряти production-збірку відкриттям dist/index.html через file://. Команда npm run preview запускає локальний HTTP-сервер для перевірки вже зібраного dist.

vite preview призначений для локальної перевірки build і не є production-сервером.



Комітимо зміни

```
svarog@svarog:~/projects/web$ git add .
svarog@svarog:~/projects/web$ git commit -m "Lb6: project restructuring on vite"
[lb6 0ba4f92] Lb6: project restructuring on vite
 14 files changed, 2263 insertions(+), 438 deletions(-)
 create mode 100644 package-lock.json
 create mode 100644 package.json
 rename {assets => public/assets}/images/placeholder.avif (100%)
 rename {assets => public/assets}/images/placeholder.jpg (100%)
 rename {assets => public/assets}/images/placeholder.webp (100%)
 rename {assets/images => public}/favicon.ico (100%)
 create mode 100644 src/main.js
 create mode 100644 src/styles/main.scss
 delete mode 100644 styles.css
 create mode 100644 vite.config.js
svarog@svarog:~/projects/web$ git push -u origin lb6
```

Наступним кроком розіб’ємо наші стилі на окремі partials та конвертуємо CSS в SCSS.

Так як стилі займають багато місця, то з прикладом моєї реалізації можна ознайомитися у посиланні на гілку лабораторної роботи, яке знаходиться у “Додаткові матеріали до лабораторної роботи №6”.

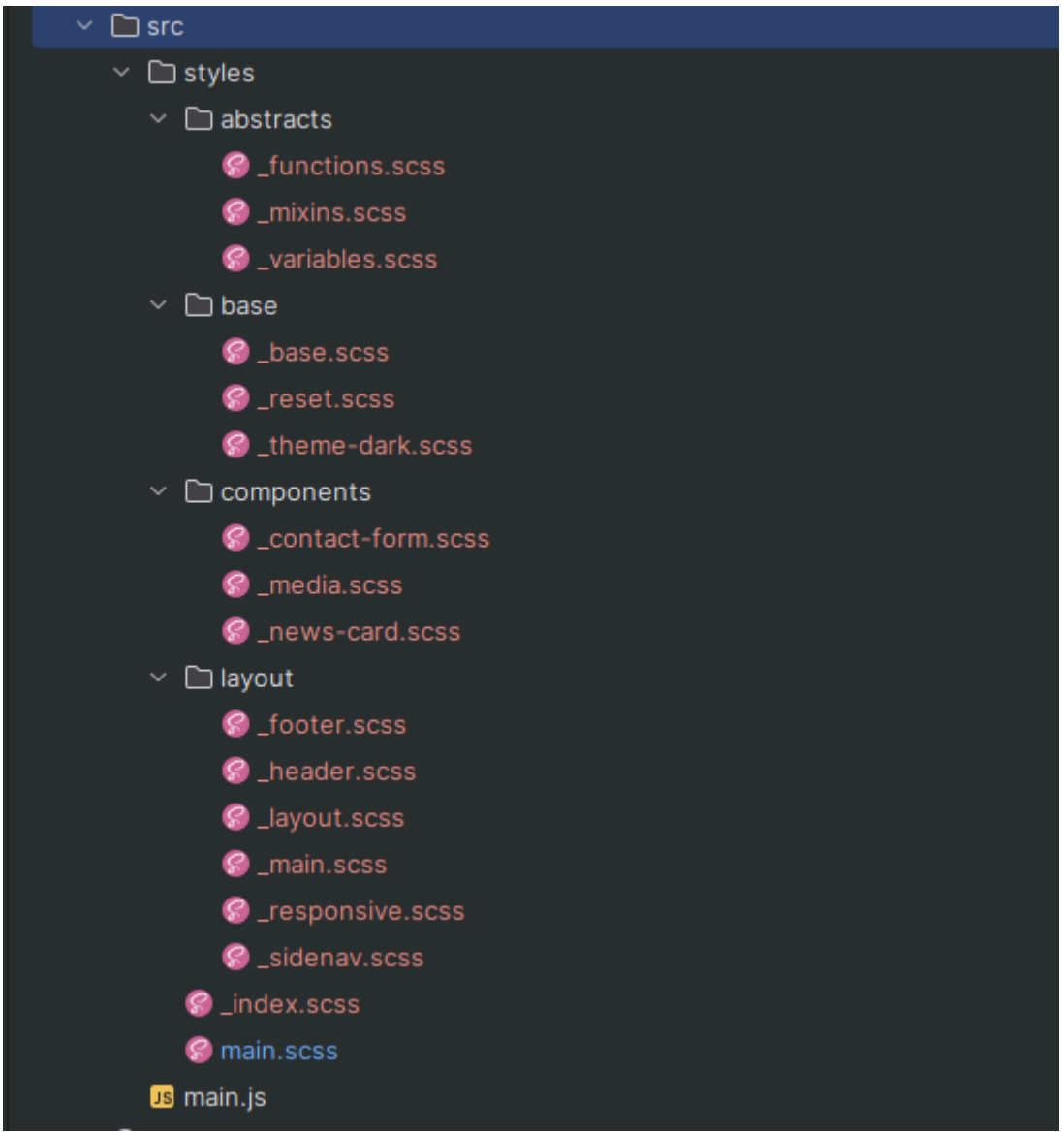
Але розглянемо структуру та зону відповідальності partial (звісно від проєкту до проєкту структура може відрізнятися, також грає роль вже наявна організація коду, яка може не зовсім відповідати стандартам).

У теці src/styles створимо 4 теки: abstracts для змінних, міксинів, функцій; base для скидання стилів, використання SCSS-змінних для CSS-змінних (--root), теми, задання поведінки базових тегів (не плутаємо зі скиданням стилів); layout для основних частин шаблону (зазвичай тих, що можуть використовуватися у декількох сторінках при MPA або за логічними блоками сторінки для SPA), components для дрібних блоків і елементів сторінок (як правило зазвичай також повторюються на різних сторінках або декілька разів у межах однієї).

Також створюємо partial-файл _index.scss для єдиної точки підключення abstracts через @forward.

У main.scss підключаємо всі файли стилів @use (за виключенням abstracts, їх підключаємо через @use './_index' as ui;).

Ось як може виглядати структура (я не проти, якщо ви використаєте свою).



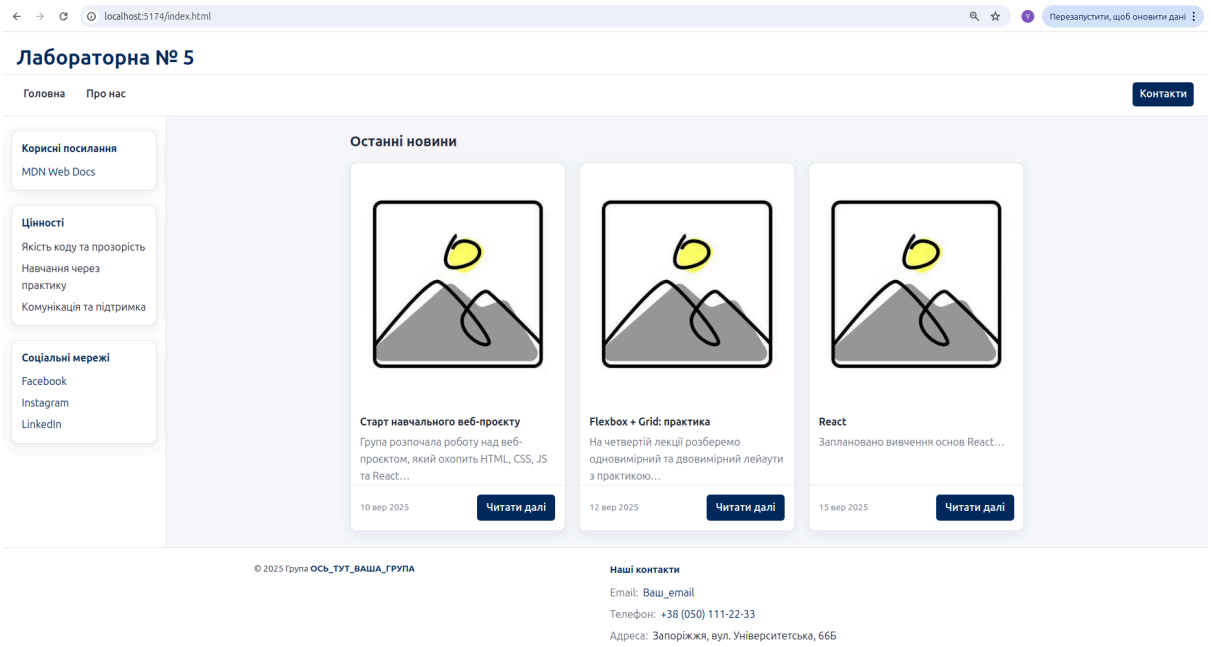
Також хочу акцентувати увагу на способи підключення стилів. Раніше використовувався @import для всього, але зараз він застарілий та не рекомендується для використання. Замість нього використовуємо вже згадані @forward та @use. Але у чому різниця? Тут також все просто, якщо пам’ятати просто їх призначення: @use для використання тут і зараз (тобто у поточному файлі потрібні дані (змінні, міксіні, просто загальне підключення тощо) з іншого); @forward, як корисна заготовка, методом групування декількох partial дозволяє їх відразу підключити до потрібного файлу (тому у нього зазвичай і підключається лише abstracts, бо тільки там тримаємо змінні, міксіни тощо).

Вносимо необхідні зміни та комітимо їх.

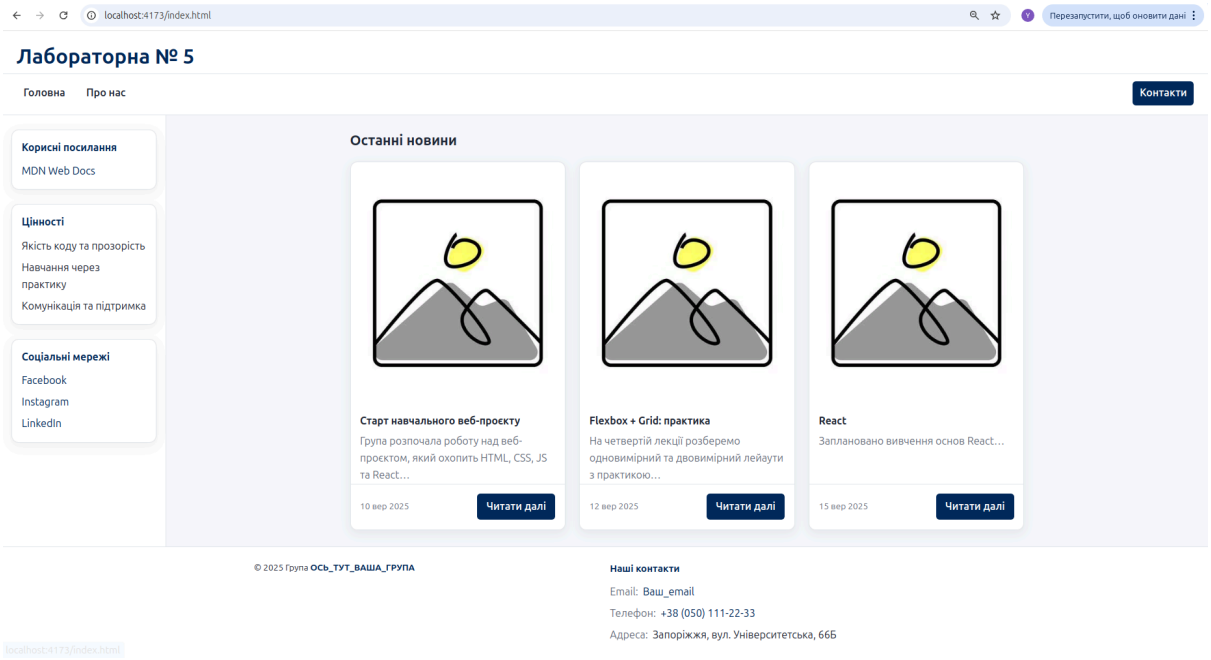
```
svarog@svarog:~/projects/web$ git add .
svarog@svarog:~/projects/web$ git commit -m "Lb6: changing the style structure, converting CSS to SCSS, updating main.scss"
[lb6 31befc8] Lb6: changing the style structure, converting CSS to SCSS, updating main.scss
 17 files changed, 464 insertions(+), 421 deletions(-)
 create mode 100644 src/styles/_index.scss
 create mode 100644 src/styles/abstracts/_functions.scss
 create mode 100644 src/styles/abstracts/_mixins.scss
 create mode 100644 src/styles/abstracts/_variables.scss
 create mode 100644 src/styles/base/_base.scss
 create mode 100644 src/styles/base/_reset.scss
 create mode 100644 src/styles/base/_theme-dark.scss
 create mode 100644 src/styles/components/_contact-form.scss
 create mode 100644 src/styles/components/_media.scss
 create mode 100644 src/styles/components/_news-card.scss
 create mode 100644 src/styles/layout/_footer.scss
 create mode 100644 src/styles/layout/_header.scss
 create mode 100644 src/styles/layout/_layout.scss
 create mode 100644 src/styles/layout/_main.scss
 create mode 100644 src/styles/layout/_responsive.scss
 create mode 100644 src/styles/layout/_sidenav.scss
svarog@svarog:~/projects/web$ git push -u origin lb6
Перерахування об’єктів: 29, готово.
Підрахунок об’єктів: 100% (29/29), готово.
Дельта компресія з використанням до 12 потоків
Компресія об’єктів: 100% (25/25), готово.
```

Також відразу перевіряємо роботу на dev та prod

Dev-сервер

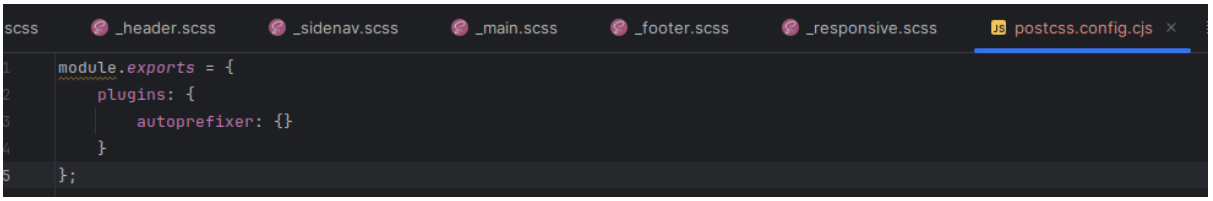


Prod (preview)

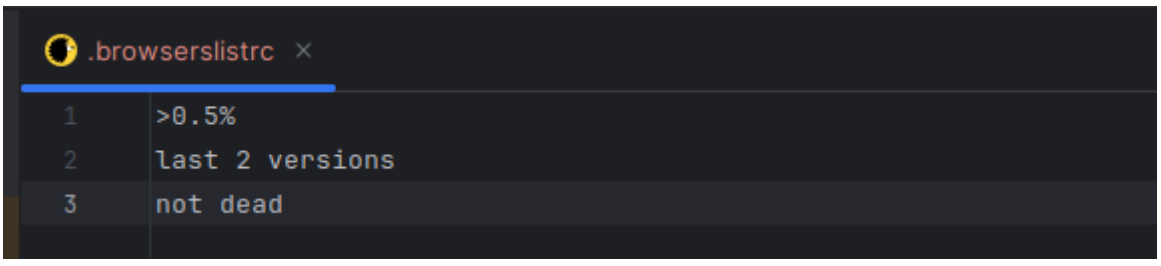


Також не варто забувати про PostCSS/Autoprefixer + Browserslist. Для них створимо файли з дефолтними налаштуваннями, а саме:

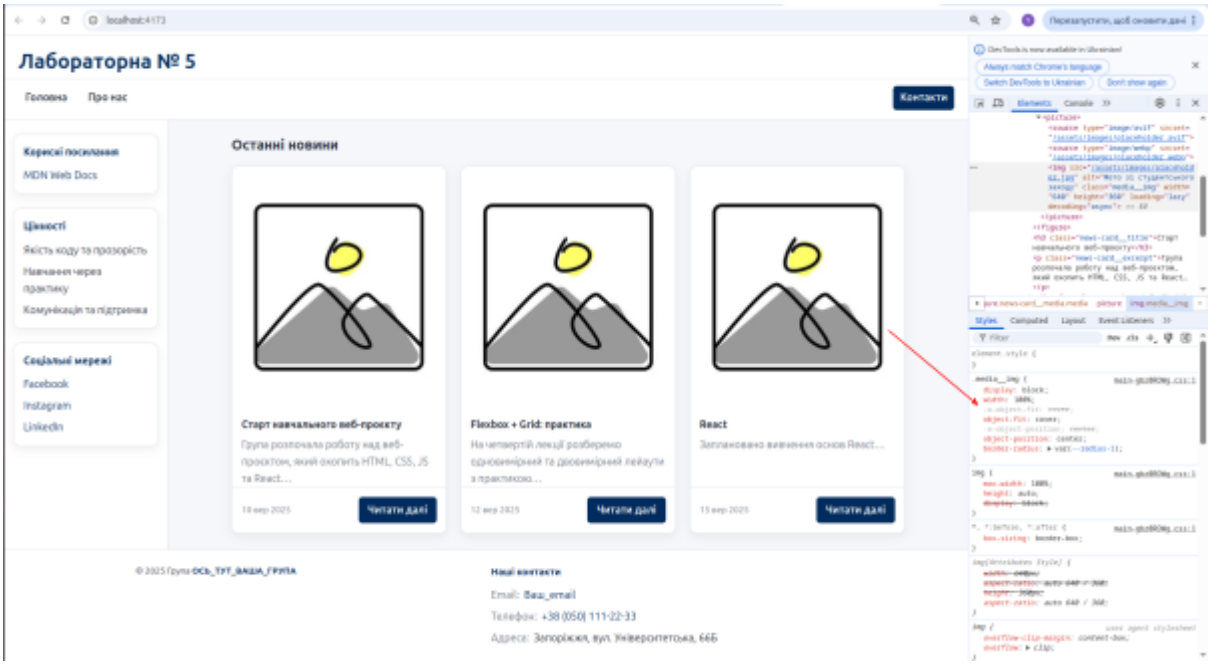
1) postcss.config.cjs (у корені проєкту) у якому підключимо плагін autoprefixer (можна підключити і у vite.config.js)



2) .browserslistrc (у корені проєкту) щоб прописати список браузерів (альтернативно можна в у package.json)



Запускаємо та перевіряємо роботу



Комітимо зміни

```
Невідстежувані файли:
  (скористайтесь "git add <файл>...", щоб додати до майбутнього коміту)
        .browserslistrc
        postcss.config.cjs

не додано жодних змін до коміту (скористайтесь "git add" та/або "git commit -a")
svarog@svarog:~/projects/web$ git add .
svarog@svarog:~/projects/web$ git commit -m "Lb6: add postcss/autoprefixer + browserslist"
[lb6 17c1f43] Lb6: add postcss/autoprefixer + browserslist
 3 files changed, 8 insertions(+), 1 deletion(-)
 create mode 100644 .browserslistrc
 create mode 100644 postcss.config.cjs
svarog@svarog:~/projects/web$ git push -u origin lb6
Перерахування об’єктів: 13, готово.
Підрахунок об’єктів: 100% (13/13), готово.
Дельта компресія з використанням до 12 потоків
Компресія об’єктів: 100% (7/7), готово.
Запис об’єктів: 100% (8/8), 780 байтів | 780.00 КіБ/с, готово.
Всього 8 (дельта 3), повторно використано 0 (дельта 0), повторно використано пакунків 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/SvarogWeb/web.git
   31befc8..17c1f43  lb6 -> lb6
гілку "lb6" налаштовано на відстежування "origin/lb6".
svarog@svarog:~/projects/web$
```

І залишився останній великий пункт цієї роботи, це розбиття на HTML-partials. Для цього спочатку встановимо необхідний плагін (я використовую vite-plugin-handlebars, можете взяти інший).

```
svarog@svarog:~/projects/web$ npm i -D vite-plugin-handlebars
```

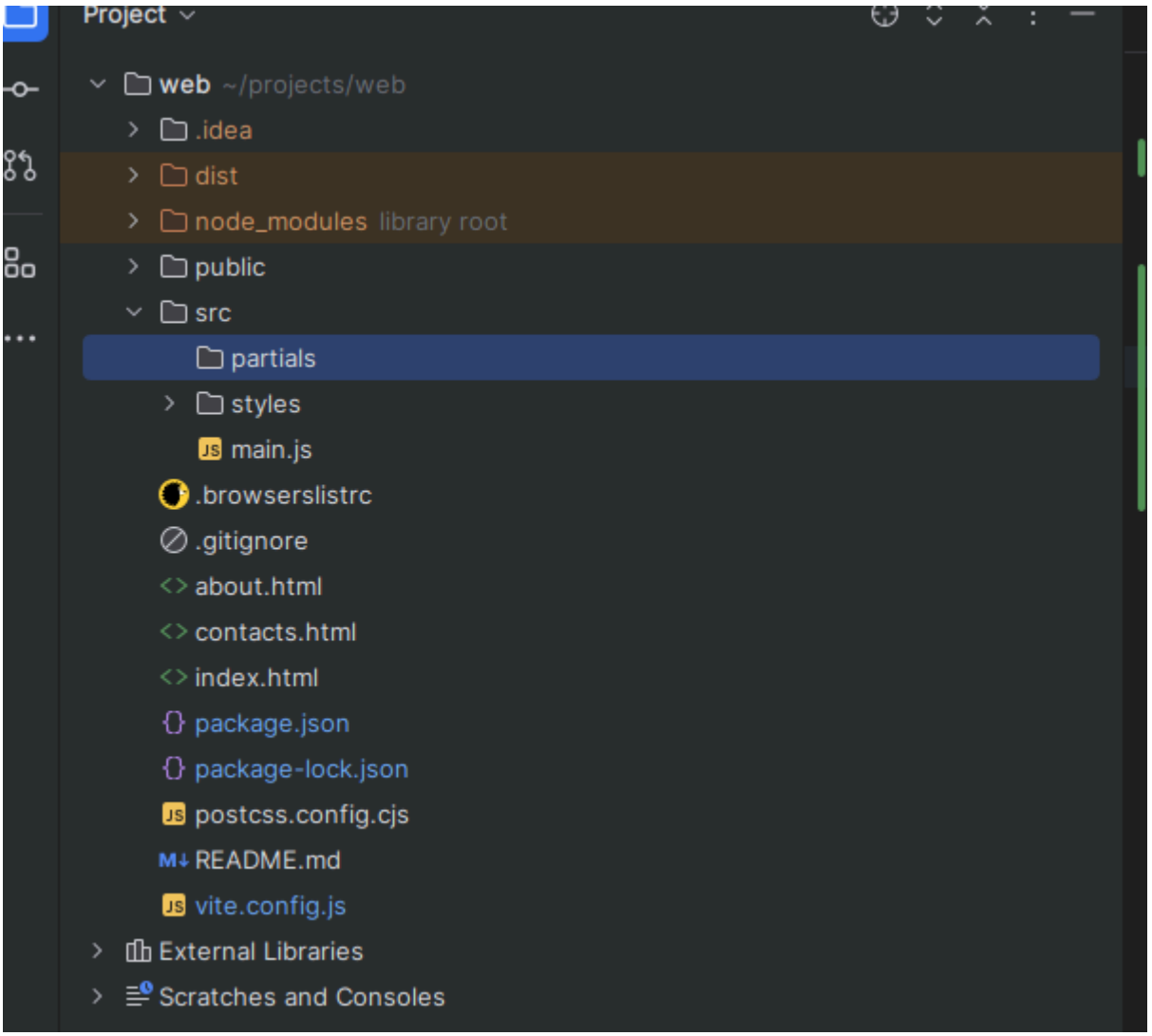
Прописуємо плагін у vite.config.js. Вказуємо шлях, де будуть зберігатися HTML-partials, а також вказуємо глобальні змінні через context, їх будемо прокидувати в title і h1.

```
export default defineConfig( config: { no usages  Yaroslav Kryvyi *
    input: {
        main: resolve(import.meta.dirname, 'index.html'),
        about: resolve(import.meta.dirname, 'about.html'),
        contacts: resolve(import.meta.dirname, 'contacts.html'),
    },
    resolve: { alias: { '@': resolve(import.meta.dirname, 'src') } },
    plugins: [
        handlebars( {context, reloadOnPartialChange = true, compileOptions, runtimeOptions, partialDirectory, helpers}: {
            partialDirectory: resolve(
                import.meta.dirname,
                'src',
                'partials'
            ),

            context: {
                siteName: 'Лабораторна 6',
                labName: 'Лабораторна № 6',
            },

            reloadOnPartialChange: true,
        }),
    ],
});
```

Створюємо відповідну теку у src



Створюємо .hbs файли для тих елементів, які ми повторно використовуємо на декількох сторінках, а також layout



Переносимо aside. У ньому ми не використовуємо змінних, тому залишаємо без змін.

aside.hbs

```hbs
<aside class="sidenav">
    <section class="sidebar sidebar--links" aria-label="Корисні посилання">
        <h3 class="sidebar__title">Корисні посилання</h3>
        <ul class="sidebar__list">
            <li class="sidebar__item">
                <a class="sidebar__link" href="https://developer.mozilla.org/" target="_blank" rel="noopener noreferrer">MDN Web Docs</a>
            </li>
        </ul>
    </section>

    <section class="sidebar sidebar--values" aria-label="Цінності">
        <h3 class="sidebar__title">Цінності</h3>
        <ul class="sidebar__list">
            <li class="sidebar__item">Якість коду та прозорість</li>
            <li class="sidebar__item">Навчання через практику</li>
            <li class="sidebar__item">Комунікація та підтримка</li>
        </ul>
    </section>

    <section class="sidebar sidebar--social" aria-label="Соціальні мережі">
        <h3 class="sidebar__title">Соціальні мережі</h3>
        <ul class="sidebar__list">
            <li class="sidebar__item"><a class="sidebar__link" href="#" aria-label="Facebook">Facebook</a></li>
            <li class="sidebar__item"><a class="sidebar__link" href="#" aria-label="Instagram">Instagram</a></li>
            <li class="sidebar__item"><a class="sidebar__link" href="#" aria-label="LinkedIn">LinkedIn</a></li>
        </ul>
    </section>
</aside>
```

Переносимо footer. У нього будемо передавати локальні змінні для групи та email, тому підставляємо майбутні змінні.

footer.hbs

```hbs
<footer class="footer">
    <div class="footer__inner">
        <p class="footer__text">© 2025 Група <span class="footer__group-name">{{ studentGroup }}</span></p>


        <address class="footer__contacts" aria-label="Наші контакти">
            <h3 class="footer__contacts-title">Наші контакти</h3>
            <ul class="footer__contacts-list">
                <li class="footer__contacts-item">
                    <span class="footer__label">Email:</span>
                    <a class="footer__link" href="mailto:team@example.com">{{ studentEmail }}</a>
                </li>
                <li class="footer__contacts-item">
                    <span class="footer__label">Телефон:</span>
                    <a class="footer__link" href="tel:+380501112233">+38 (050) 111-22-33</a>
                </li>
                <li class="footer__contacts-item">
                    <span class="footer__label">Адреса:</span>
                    <span>Запоріжжя, вул. Університетська, 66Б</span>
                </li>
            </ul>
        </address>
    </div>
</footer>
```

За аналогією переносимо header. У ньому використаємо глобальну змінну labName, яку ми вже прописали у context vite.config.js.

header.hbs

```hbs
<header class="header">
    <h1 class="header__title">{{ labName }}</h1>
    <nav class="nav">
        <a href="/index.html" class="nav__link">Головна</a>
        <a href="/about.html" class="nav__link">Про нас</a>
        <a href="/contacts.html" class="nav__link nav__link--cta">Контакти</a>
    </nav>
</header>
```

Переносимо head. У ньому використовуємо дві змінні: глобальну siteName та локальну pageTitle.

contacts.html | package.json | layout.hbs | header.hbs | index.html | head.hbs | vite.config.js

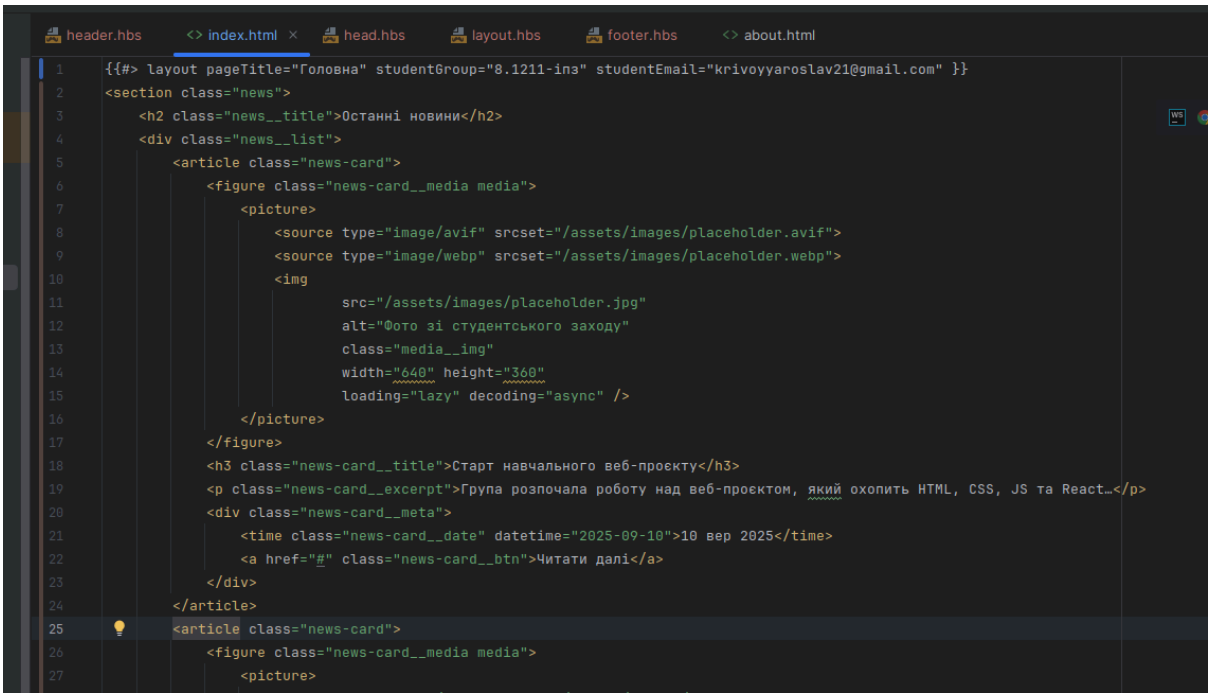
```hbs
<meta charset="UTF-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1.0"/>
<title>{{ siteName }} - {{ pageTitle }}</title>
<meta name="description" content="Головна сторінка навчального лендінгу: новини, послуги, переваги."/>
<link rel="icon" href="/favicon.ico"/>
```

Переходимо до layout, підключаємо до нього всі сторінки та вказуємо, які змінні передати. Також хочу звернути увагу, що тег <main> краще залишити у цьому файлі, так і зрозуміліше, що ми туди будемо підставляти і семантично правильно. Також studentGroup і studentEmail можна задати відразу, так як вони спільні для всіх сторінок (а краще зробити глобальними), але у межах тренування залишаємо так.

header.hbs | index.html | head.hbs | layout.hbs | footer.hbs | about.html

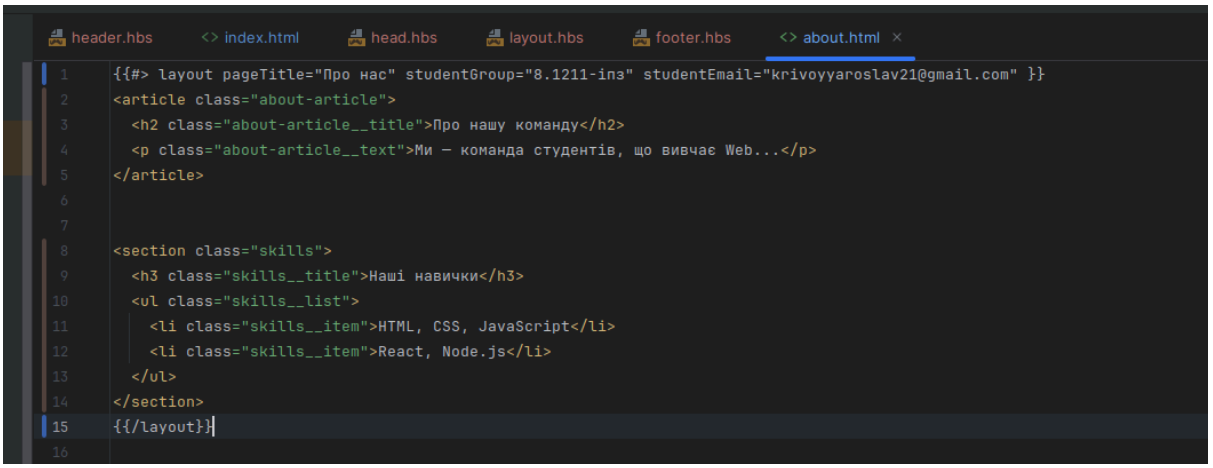
```hbs
<!DOCTYPE html>
<html lang="uk">
<head>
    {{> head pageTitle=pageTitle }}
</head>
<body>
<div class="layout">
    {{> header }}
    {{> aside }}
    <main class="main">
        {{> @partial-block }}
    </main>
    {{> footer studentGroup=studentGroup studentEmail=studentEmail}}
</div>
<script type="module" src="/src/main.js"></script>
</body>
</html>
```

Тепер переходимо до наших сторінок. Почнемо з index.html. Із-за того що ми створили так звану “рамку” (layout), який є спільним для наших сторінок, то вказувати підключення скрипта, head, body, html не потрібно. Міняємо тільки контент в main.

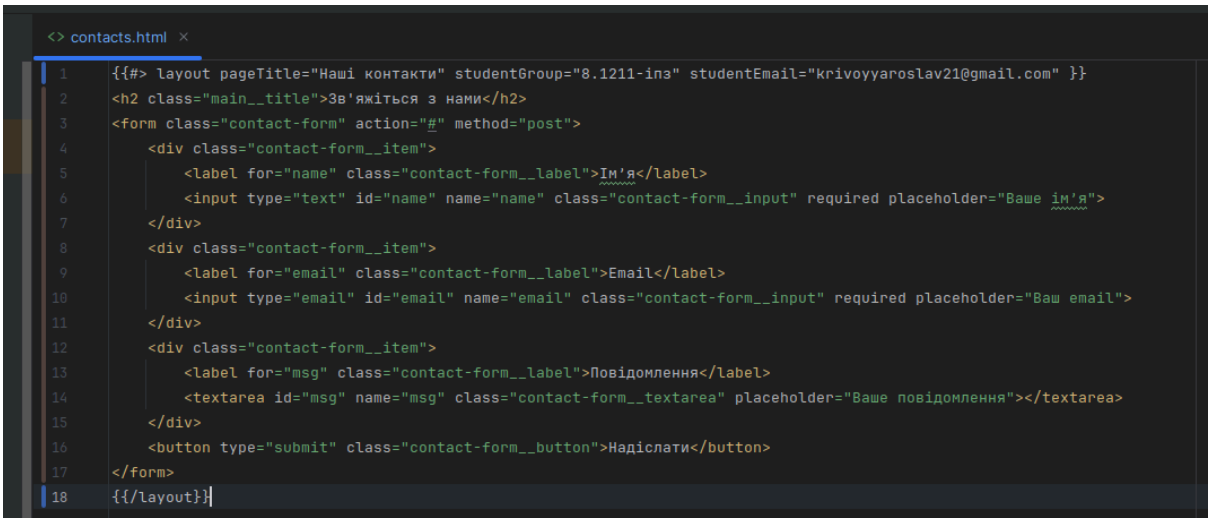


З приводу карток новин, їх також краще виносити окремо, але у межах навчального проєкту залишаємо так.

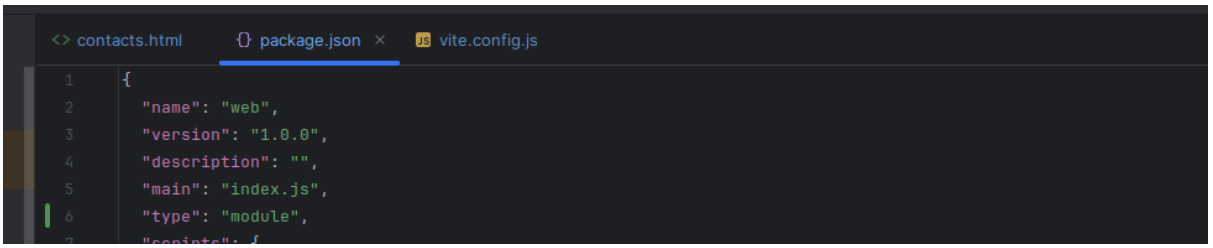
За аналогією робимо about.html



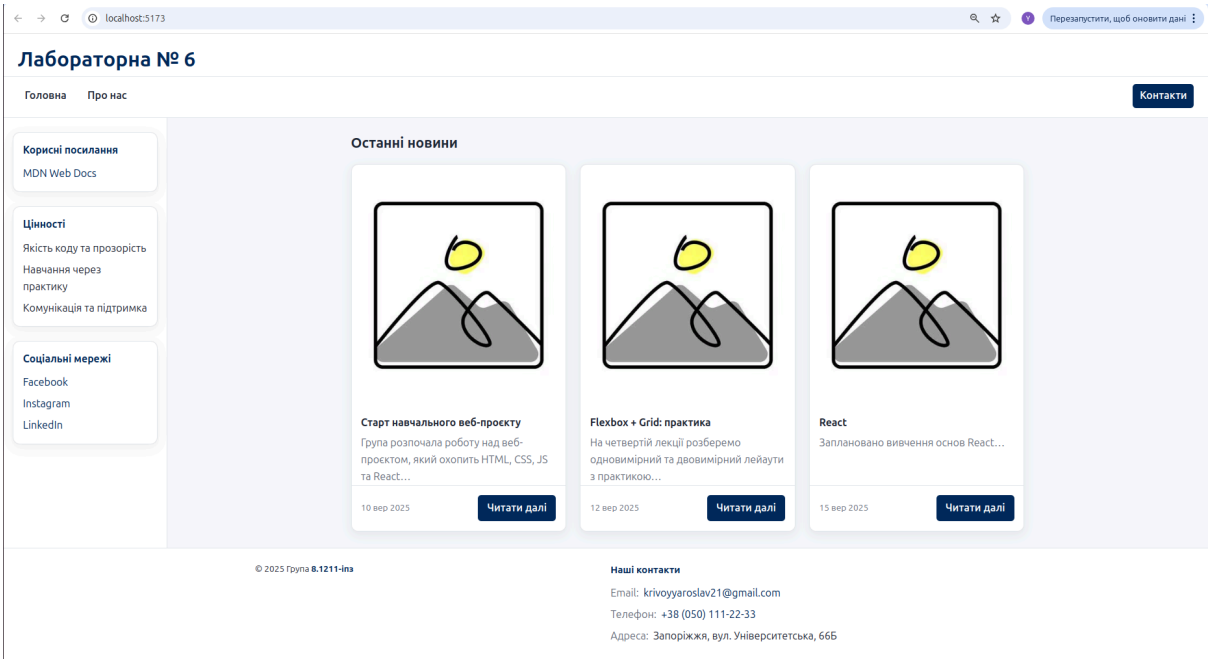
За аналогією робимо contacts.html



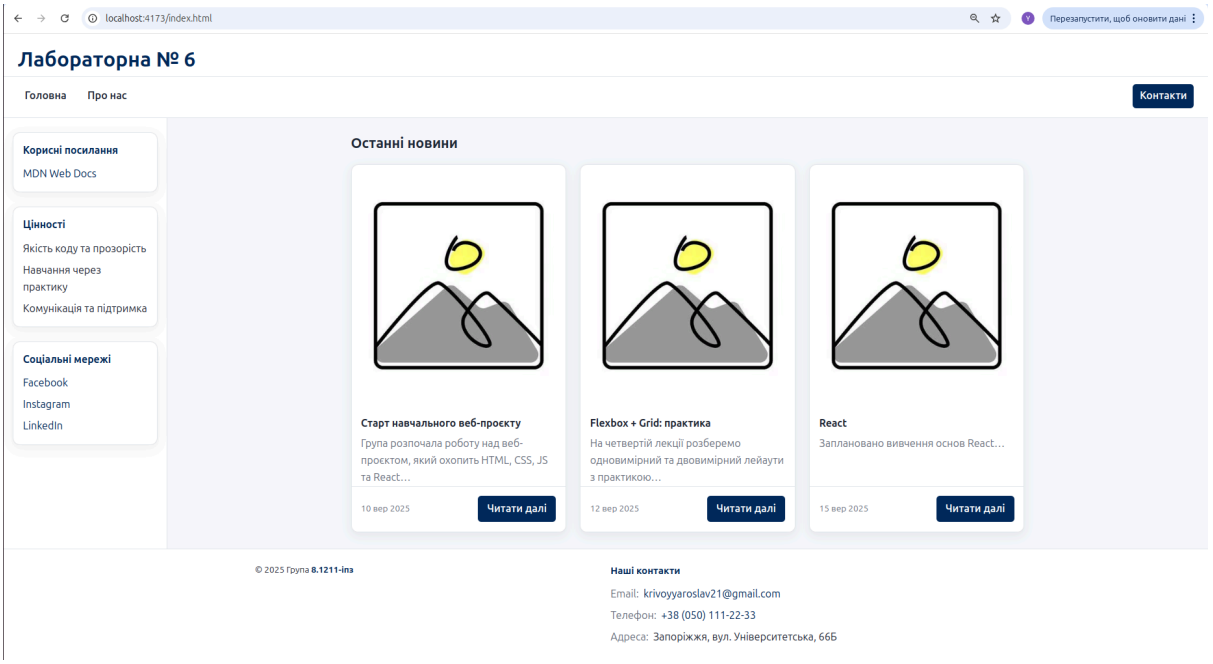
Також, так як працює на ESM, то необхідно вказати явно type: module у package.json.



Запускаємо та перевіряємо dev



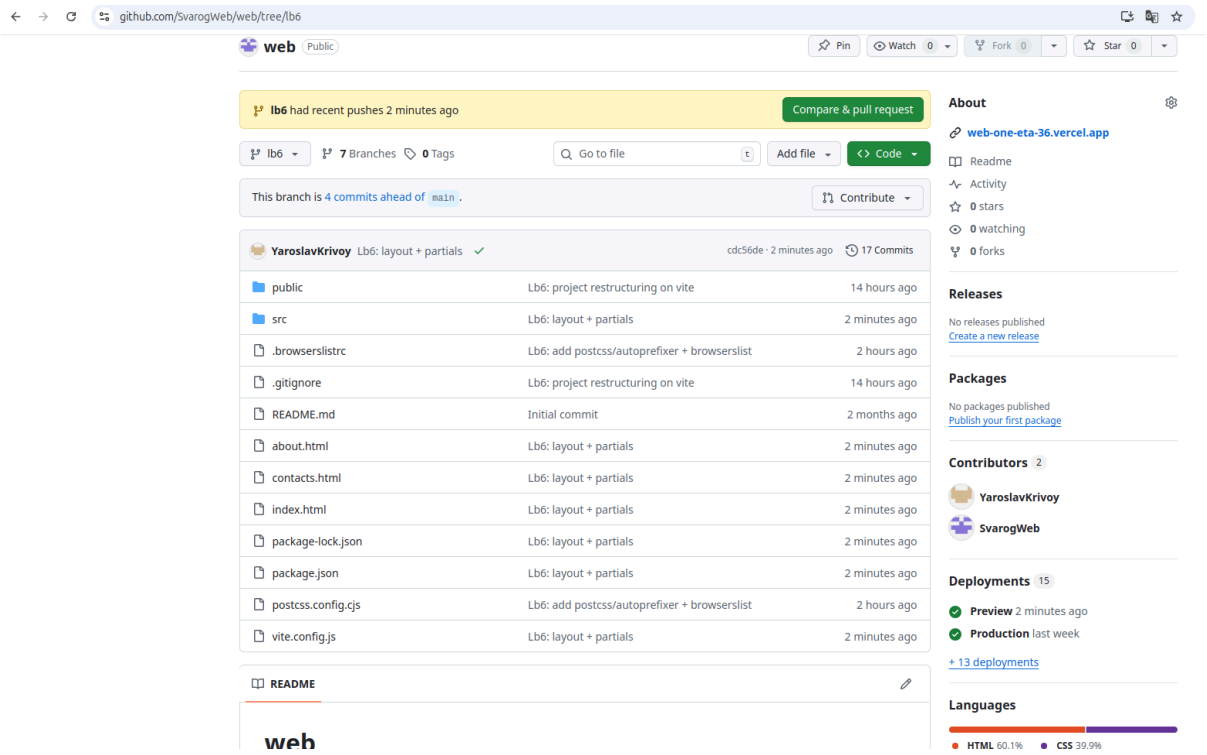
А також збираємо проєкт та знов перевіряємо у preview



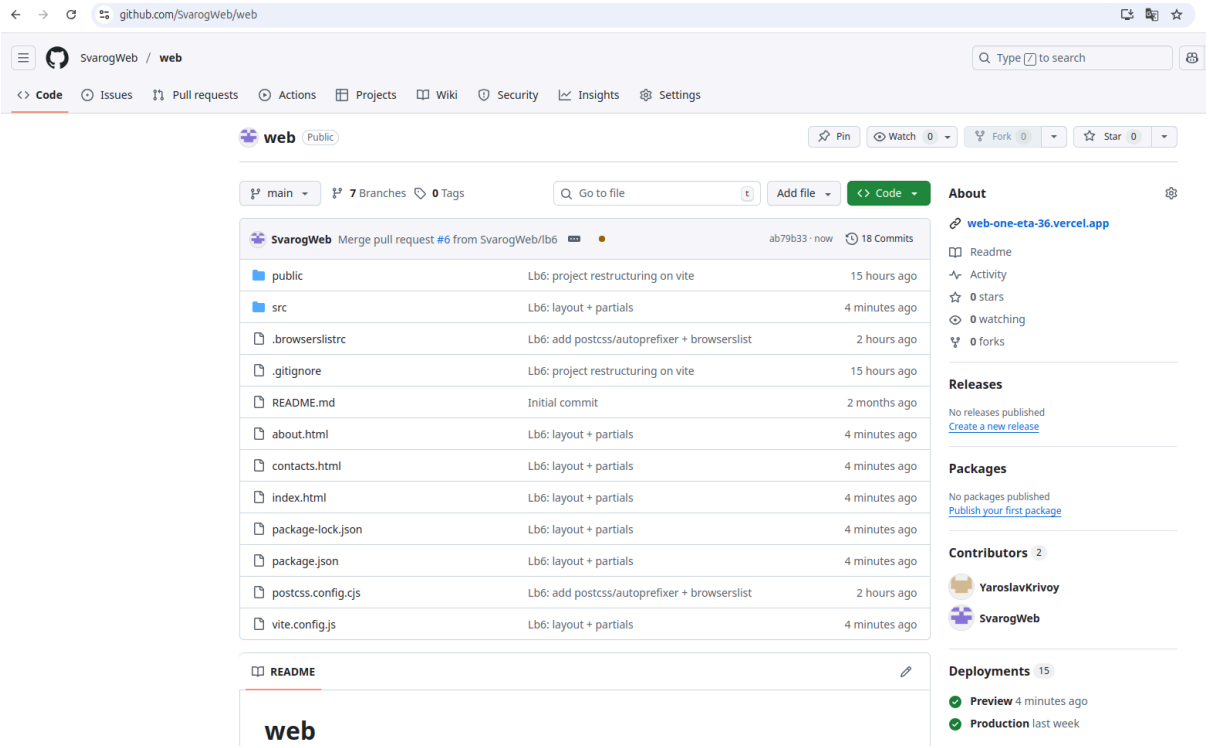
Комітимо зміни та відправляємо на віддалену гілку

```
не додано жодних змін до коміту (скористайтесь "git add" та/або "git commit -a")
svarog@svarog:~/projects/web$ git add .
svarog@svarog:~/projects/web$ git commit -m "Lb6: layout + partials"
[lb6 cdc56de] Lb6: layout + partials
 11 files changed, 771 insertions(+), 345 deletions(-)
 create mode 100644 src/partials/aside.hbs
 create mode 100644 src/partials/footer.hbs
 create mode 100644 src/partials/head.hbs
 create mode 100644 src/partials/header.hbs
 create mode 100644 src/partials/layout.hbs
svarog@svarog:~/projects/web$ git push -u origin lb6
Перерахування об'єктів: 23, готово.
Підрахунок об'єктів: 100% (23/23), готово.
Дельта компресія з використанням до 12 потоків
Компресія об'єктів: 100% (15/15), готово.
Запис об'єктів: 100% (15/15), 7.25 КіБ | 3.63 МіБ/с, готово.
Всього 15 (дельта 5), повторно використано 0 (дельта 0), повторно використано пакунків 0
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
To https://github.com/SvarogWeb/web.git
   17c1f43..cdc56de  lb6 -> lb6
гілку "lb6" налаштовано на відстежування "origin/lb6".
svarog@svarog:~/projects/web$
```

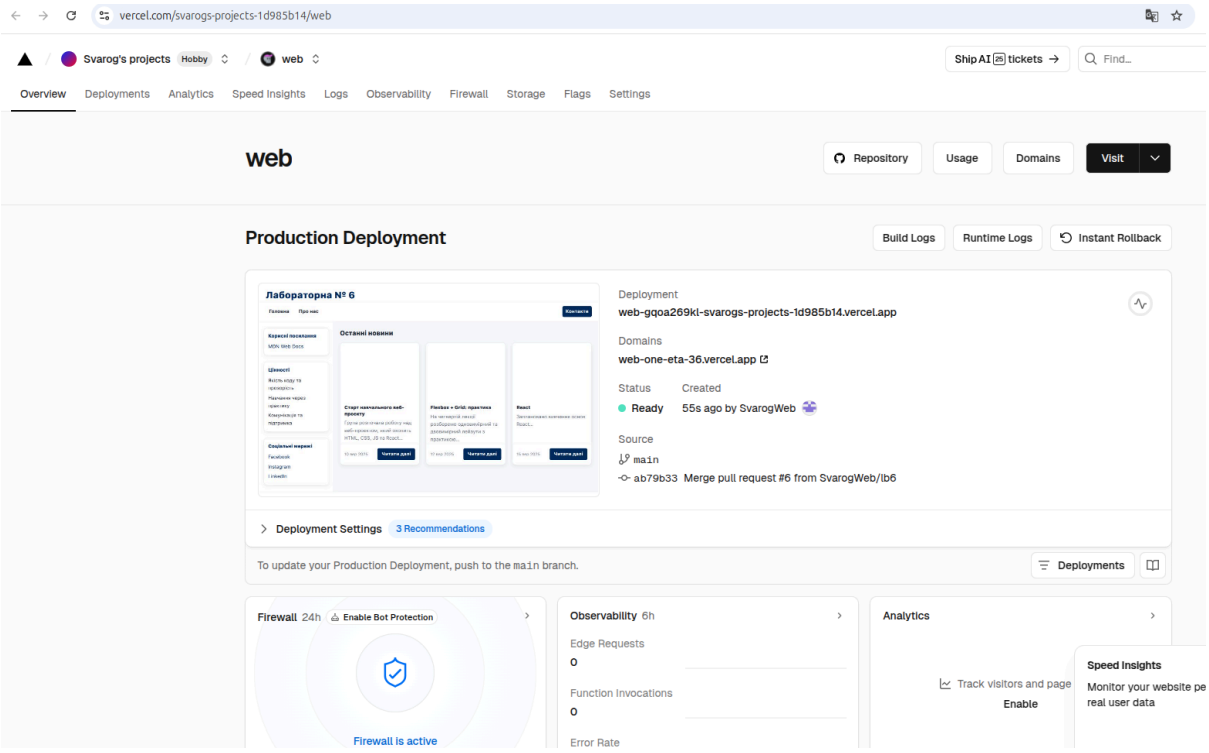
Перевіряємо віддалену гілку



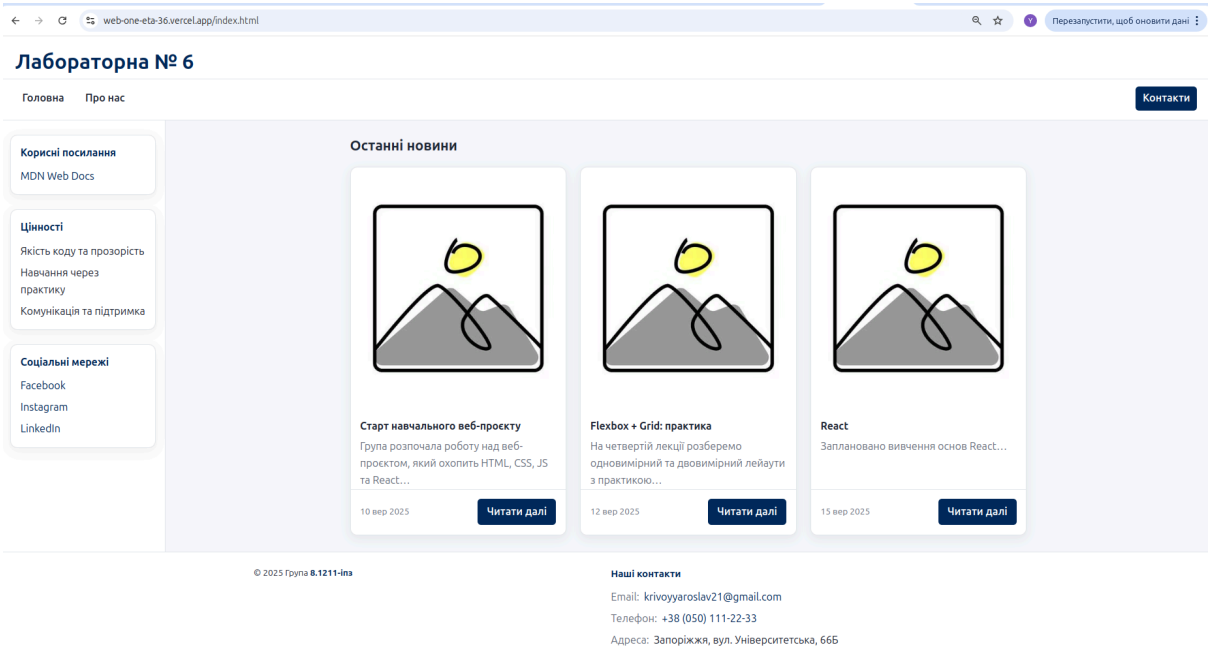
3. Відправляємо зміни на main



Перевіряємо Versel



І відображення



Переходимо на локальну гілку main та забираємо зміни з віддаленої гілки

```
svarog@svarog:~/projects/web$ git switch main
Переключено на гілку "main"
Ваша гілка не відрізняється від "origin/main".
svarog@svarog:~/projects/web$ git pull
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Розпакування об'єктів: 100% (1/1), 882 байти | 882.00 КіБ/с, готово.
Від https://github.com/SvarogWeb/web
   12f5247..ab79b33  main       -> origin/main
Оновлення 12f5247..ab79b33
Fast-forward
 .browserslistrc                          |    3 +
 .gitignore                               |   11 +-
 about.html                               |  113 ++------
 contacts.html                            |  118 ++------
 index.html                               |  210 +++++----------
 package-lock.json                        | 2352 ++++++
```

3) Як налаштувати PostCSS + Autoprefixer у Vite? Яку роль відіграє Browserslist?
4) Які переваги Sass/SCSS порівняно зі звичайним CSS? Поясніть різницю між Sass-змінними $var і CSS-змінними --var та коли які доречні;
5) Як працюють @use і @forward у SCSS?
6) Що таке HTML-partials? Чому це DRY-підхід?